

Sprint 3 Implementation Guide

Sprint 3 Implementation Guide — Booking & Communication

Project: NexGiG (venomwastaken/nexgig) — FastAPI + SQLAlchemy backend, React + Vite + TS frontend, Clerk auth **Sprint Goal:** Enable the core transaction loop (client → freelancer) and give admins account-level control.

Backlog items in scope: | ID | Item | Priority | |—|—| | PB6 | Hire/book a freelancer through the platform | High | | PB7 | In-platform messaging between freelancers and clients | High | | PB11 | Administrator management of user accounts | High |

0. Before you start

1. Pull latest `main`, confirm the stack boots per the README:

```
docker compose up          # backend + Postgres, own terminal tab
cd frontend && npm run dev   # frontend, separate tab
```

Visit `http://localhost:5173` and confirm “Backend status: ok” before touching anything.

2. Activate the backend venv for any model/migration work: `cd backend && .\venv\Scripts\Activate.ps1`.
 3. Branch off `main` per ticket: `feature/pb6-booking`, `feature/pb7-messaging`, `feature/pb11-admin-users`. Don’t bundle all three in one PR — CI and review will be painful, and PB7 has a genuine size/architecture risk the sprint plan itself flags.
 4. Recommended order: **PB6 → PB11 → PB7**. Booking creates the relationship (client↔freelancer, plus a booking/service context) that messaging will likely hang off of. PB11 is largely independent and low-risk, good to slot in parallel if you have a second developer. PB7 is the most architecturally complex — the sprint plan explicitly calls out that it may need to spill into a following sprint, so treat it as its own mini-project (see §3).
-

1. PB6 — Hire/Book a Freelancer

1.1 Data model

Add a `Booking` SQLAlchemy model in `backend/app/models/` (match whatever convention `UserAccount` and your service-listing model already use — check for an existing `models.py` vs. `models/` package before creating new files).

Suggested fields:

```
class BookingStatus(str, Enum):
    pending = "pending"
    accepted = "accepted"
    declined = "declined"
    completed = "completed"
    cancelled = "cancelled"

class Booking(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    listing_id: int = Field(foreign_key="servicelisting.id")
    client_id: int = Field(foreign_key="useraccount.id")
    freelancer_id: int = Field(foreign_key="useraccount.id")
    status: BookingStatus = Field(default=BookingStatus.pending)
    message: Optional[str] = None          # optional note from client at booking time
```

```
created_at: datetime = Field(default_factory=datetime.utcnow)
updated_at: datetime = Field(default_factory=datetime.utcnow)
```

Notes: - Reuse whatever table names Alembic already generated for your existing models (`useraccount` , `servicelisting` , etc.) — check `backend/alembic/versions/` for the actual table names rather than assuming. - `client_id` and `freelancer_id` both point at `UserAccount` ; make sure your FK naming doesn't collide (SQLModel/Alembic autogenerate sometimes needs manual constraint naming when a table has two FKs to the same table). - Don't duplicate anything Clerk owns (see README's auth note) — `client_id` / `freelancer_id` should reference your internal `UserAccount.id` , not a Clerk ID directly, if that's the existing convention elsewhere in the codebase.

1.2 Migration

```
cd backend
.\env\Scripts\Activate.ps1
alembic revision --autogenerate -m "add booking table"
```

Before `alembic upgrade head` , run through the repo's own checklist: - `venv` active (`(venv)` in prompt) - `docker compose up` running (Postgres must be reachable) - `DATABASE_URL` in `backend/.env` points to `localhost` , not `db` - **Manually review** the generated migration file — SQLModel autogenerate is known to need manual fixes here, especially with the double-FK-to-UserAccount situation above.

1.3 Backend endpoints

Add a `bookings` router (mirror however existing routers are structured/mounted in `app/main.py`): - `POST /bookings` — client creates a booking against a `listing_id` . Validate the listing exists and is approved (ties into PB10 from Sprint 2 — a listing must have passed admin approval before it's bookable). - `GET /bookings/me` — bookings where the current user is either client or freelancer (filter by role/query param). - `PATCH /bookings/{id}/status` — freelancer accepts/declines; either party can cancel. Enforce that only the freelancer on the booking can move `pending` → `accepted/declined` , and only the client can `cancel` a `pending` booking. Return 403 otherwise. - Auth: every route pulls the current user from the verified Clerk JWT (`PyJWKClient` , as already set up) — never trust a client-supplied user ID in the request body for who's booking.

1.4 Frontend

- A “Book this freelancer” CTA on the service-listing detail page → opens a booking form/modal (optional message field) → `POST /bookings` .
- A “My Bookings” view, split into two tabs if the same user can be both a client and freelancer: “Booking requests I've sent” and “Booking requests I've received” (with accept/decline actions on the latter).
- Handle the 401/403 cases from the backend explicitly in the UI (e.g. redirect to sign-in via Clerk if unauthenticated) rather than showing a raw error.

1.5 Tests / CI

- Backend: unit tests for status-transition rules (can't accept a `declined` booking, can't book your own listing, etc.) and the 403 authorization boundaries above.
- Confirm CI passes on your PR branch itself — remember the repo's CI quirk: GitHub Actions runs the workflow file **from your branch**, so if you ever touch `.github/workflows/ci.yml` , you need a fresh commit on the branch, not just a re-run, to pick it up.

2. PB11 — Administrator Management of User Accounts

2.1 Scope

Give admins the ability to view all accounts and take action: suspend/reactivate an account, and (likely) change a role/flag (e.g. freelancer vs client vs admin) if your `UserAccount` model has one. Do **not** build password reset, email verification, or login management here — per the README, Clerk is the identity source of truth and already owns those.

2.2 Data model

If there isn't already a status field on `UserAccount`, add one:

```
class AccountStatus(str, Enum):
    active = "active"
    suspended = "suspended"

# on UserAccount:
status: AccountStatus = Field(default=AccountStatus.active)
```

Migration via the same Alembic flow as §1.2.

2.3 Backend endpoints

- GET `/admin/users` — paginated list, filterable by status/role. Restrict to admin role only.
- PATCH `/admin/users/{id}/status` — suspend/reactivate.
- Add an admin-only dependency (e.g. `require_admin`) if one doesn't already exist from PB10 (listing approval) in Sprint 2 — reuse it rather than re-implementing.
- Suspension should have a real effect: enforce a check (middleware or per-route dependency) that blocks a suspended user's write actions (posting listings, booking, messaging) even though their Clerk session is still valid. This is the part most likely to be missed — a suspended flag in the DB does nothing unless it's actually checked.

2.4 Frontend

- An admin dashboard page (likely already exists from PB10's listing-approval work — extend it rather than starting fresh) with a user table: name, role, status, and a suspend/reactivate action.
- Gate the route client-side on the admin role, but treat that as UX only — the real enforcement is server-side in §2.3.

2.5 Tests

- Non-admin gets 403 on all `/admin/*` routes.
- Suspended user is blocked from the specific write actions you decided to gate.

3. PB7 — In-Platform Messaging

This is the item the sprint plan flags as a real complexity risk. Scope it deliberately rather than trying to ship real-time chat in one pass.

3.1 Recommended MVP scope for this sprint

Ship synchronous, poll-based messaging first; treat WebSocket/real-time push as a stretch goal or a follow-up sprint item, matching the plan's own note that this may need to spill over.

3.2 Data model

```
class Conversation(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    booking_id: Optional[int] = Field(default=None, foreign_key="booking.id")
    client_id: int = Field(foreign_key="useraccount.id")
    freelancer_id: int = Field(foreign_key="useraccount.id")
    created_at: datetime = Field(default_factory=datetime.utcnow)

class Message(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    conversation_id: int = Field(foreign_key="conversation.id")
    sender_id: int = Field(foreign_key="useraccount.id")
    body: str
    created_at: datetime = Field(default_factory=datetime.utcnow)
    read_at: Optional[datetime] = None
```

Tying `Conversation` to a `Booking` (nullable, since you may want to allow pre-booking questions) keeps the messaging scoped and gives you a natural permission boundary: only the two parties on the booking can see the conversation.

3.3 Backend endpoints

- `POST /conversations` — start or fetch-existing conversation between a client and freelancer (optionally tied to a booking/listing).
- `GET /conversations/me` — list conversations for current user, with last-message preview.
- `GET /conversations/{id}/messages` — paginated message history; 403 if the current user isn't a participant.
- `POST /conversations/{id}/messages` — send a message; 403 for non-participants; block if either party's account is suspended (ties into §2.3).
- If you scope to polling: the frontend polls `GET /conversations/{id}/messages?since=<timestamp>` on an interval. If you attempt WebSockets, isolate it behind a feature flag so PB6/PB11 aren't blocked on it landing.

3.4 Migration

Same Alembic flow as above — two new tables in one migration is fine, but double-check the FK ordering in the autogenerated file (Alembic needs `conversation` created before `message` if it references it).

3.5 Frontend

- Inbox view: conversation list with last message + timestamp.
- Thread view: message list + composer. If polling, keep the interval modest (e.g. 5-10s) and pause polling when the tab isn't focused.
- Entry points: "Message" button on a booking, and/or on a freelancer's profile.

3.6 Tests

- Authorization: only participants can read/write a conversation.
- Suspended-user block (shared logic with §2.3 — write it once, use in both places).
- Pagination correctness on message history for a conversation with many messages.

4. Cross-cutting checklist before opening each PR

- ☐ Branch is `feature/<short-description>` off current `main`
- ☐ alembic upgrade head runs clean from a fresh `docker compose up` + migration history
- ☐ Reviewed the autogenerated migration file by hand
- ☐ No new fields on `UserAccount` duplicate anything Clerk owns
- ☐ Every new route authenticates via the verified JWT, not a client-supplied user ID
- ☐ `npm install <pkg> --save-exact` used for any new frontend dependency
- ☐ Vite left pinned to `5.4.x` — no upgrade past that without team sign-off
- ☐ CI is green **on your branch** (push a real commit if you touched the workflow file itself)
- ☐ PR into `main`, no direct pushes

5. Suggested order of work within the sprint

1. PB6 backend (model, migration, endpoints) → PB6 frontend → PR/merge.
2. PB11 backend + frontend in parallel with PB6 if you have two developers, since it's largely independent. → PR/merge.
3. PB7: data model + MVP polling endpoints → basic inbox/thread UI → PR/merge. If real-time/WebSocket delivery doesn't fit in the sprint, ship polling and open a follow-up ticket for the upgrade rather than blocking the sprint on it — this matches the plan's own note that Sprint 3 may need to span additional sprints.